

Essa ERP — AWS Database Connection Guide

Provisioning and connecting Amazon RDS for PostgreSQL in Mumbai (ap-south-1)

Prepared for Essa Garments Private Limited · 4 September 2026 · Document 1 of 2

This document is the step-by-step for standing up the Essa database on AWS and pointing the application at it. It is written against the actual code in `essa-intake` — the warehouse app (FastAPI), the retail shop (Flask, mounted at `/pos`) and the phone app all share one database, and a few of the steps below exist only because of how that code reads its connection string. Pricing is in the companion document.

BEFORE YOU BEGIN

Creating the RDS instance is a billable action and the "Create database" click is yours to make — this guide takes you to that button and through everything after it. Set a billing budget first (Step 0) so a mis-sized instance cannot run for a month unnoticed.

0 Prerequisites and account setup

- AWS account with billing set up.** For an Indian entity this is an AISPL account invoiced in INR with 18% GST. Add Essa's GSTIN (`33AADCE6591N1Z7`) under *Billing* → *Tax settings* before the first invoice so the GST is claimable as input credit.
- Region: Asia Pacific (Mumbai)** `ap-south-1` . Pick it in the top-right region selector and never change it mid-build — resources in different regions cannot talk to each other privately, and cross-region traffic is billed.
- Do not build as the root user.** Create an IAM user (or IAM Identity Center user) with `AmazonRDSFullAccess` + `AmazonEC2FullAccess` , enable MFA, and use that.
- Set a budget alarm.** *Billing* → *Budgets* → *Create budget* → *Monthly cost*. Set the amount from the plan you choose in Document 2 and add an email alert at 80% and 100%.
- Install a Postgres client locally** — `psql` (from the PostgreSQL client tools) or a GUI such as pgAdmin or DBeaver. You need version 16 or newer.

1 Create the RDS PostgreSQL instance

Console → **RDS** → **Databases** → **Create database**. Choose *Standard create* (the Easy create defaults are not the ones you want). Then work down the form:

Setting	Value for Essa	Why
Engine type	PostgreSQL	The app is SQLAlchemy + psycopg2; no other engine works.
Engine version	PostgreSQL 16.x or 17.x (latest minor)	Staying on a supported major version avoids RDS Extended Support charges later.

Setting	Value for Essa	Why
Templates	Production (or Dev/Test for a pilot)	Only changes the defaults below; you can override every one.
DB instance identifier	essa-prod-db	Becomes part of the endpoint hostname.
Master username	essa_admin	Avoid postgres; this account is for administration only, not for the app.
Credentials management	Self managed, or AWS Secrets Manager	Secrets Manager adds a small monthly cost but gives you rotation and no password in a config file.
Master password	Long, and no @ : / ? # % & characters	Those characters break a URL-form connection string unless percent-encoded. Save it in your password manager now.
Instance class	db.t4g.small (see Document 2)	Graviton; roughly half the hourly rate of the equivalent t3 class in Mumbai.
Storage type	General Purpose SSD (gp3)	3,000 IOPS and 125 MB/s included at no extra charge.
Allocated storage	100 GiB	Also sets your free backup allowance — see Document 2.
Storage autoscaling	Enabled, max 500 GiB	A full disk takes the database offline; autoscaling prevents it.
Availability	Single-AZ to start; Multi-AZ for HA	Multi-AZ exactly doubles both instance and storage cost.
Compute resource	Don't connect to an EC2 compute resource	Wire the security groups yourself in Step 2 — clearer and reusable.
VPC	The VPC the app server will live in	The database and app server must share a VPC to talk privately.
Public access	No	See the note below if you need to reach it from the office.
VPC security group	Create new: essa-db-sg	Configured in Step 2.
Database port	5432	Default.
Database authentication	Password authentication	The app sends a username and password.
Initial database name	essa	Easy to miss. Leave it blank and RDS creates the instance with no database in it.
Backup retention	7 days (30 for production)	Automated point-in-time recovery.
Encryption	Enabled (default AWS KMS key)	Free, and cannot be turned on after creation.
Performance Insights	Enabled, 7-day retention	Free at 7 days; the fastest way to find a slow query later.
Auto minor version upgrade	Enabled	Security patches during your maintenance window.

Setting	Value for Essa	Why
Deletion protection	Enabled	Stops an accidental delete of the production database.

Click **Create database**. It takes 5–10 minutes to reach *Available*. When it does, open the instance and copy the **Endpoint** — it looks like `essa-prod-db.abcd1234efgh.ap-south-1.rds.amazonaws.com`.

PUBLIC ACCESS — THE HONEST VERSION

"Public access: No" is right for production, but it also means you cannot connect from your laptop. The two clean ways in are a small bastion EC2 instance reached over AWS Systems Manager Session Manager (no open SSH port, no key files), or an SSM port-forward straight to the database. If you must set public access to Yes for the first week of setup, restrict the security group to your office's public IP with a `/32` mask and turn it off again afterwards. Never open 5432 to `0.0.0.0/0`.

2 Networking — who is allowed to connect

A new RDS instance rejects everything by default. Two rules make it work:

RULE A — THE APPLICATION SERVER

In **EC2** → **Security Groups** → `essa-db-sg` → **Inbound rules** → **Edit**:

Type	Protocol	Port	Source
PostgreSQL	TCP	5432	The <i>security group ID</i> of the app server (e.g. <code>sg-0abc...</code> , named <code>essa-app-sg</code>)

Use the security group as the source, not an IP address. The app server's private IP changes when it is replaced; its security group does not.

RULE B — ADMINISTRATIVE ACCESS (TEMPORARY)

Type	Protocol	Port	Source
PostgreSQL	TCP	5432	Your office public IP as <code>x.x.x.x/32</code> , or the bastion's security group

CONNECTING THROUGH SSM INSTEAD OF OPENING A PORT

```
aws ssm start-session \
  --target i-0bastioninstanceid \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{"host":["essa-prod-db.abcd1234efgh.ap-south-1.rds.amazonaws.com"],"portNumber":
["5432"],"localPortNumber":["5432"]}' \
  --region ap-south-1
```

The database is then reachable at `localhost:5432` on your machine, with nothing exposed to the internet.

3 First connection test

Before touching the application, prove the network path works. From your laptop or the bastion:

```
# quick test – encrypted, but does not verify the server certificate
psql "host=essa-prod-db.abcd1234efgh.ap-south-1.rds.amazonaws.com \
port=5432 dbname=essa user=essa_admin sslmode=require"
```

For a properly verified connection, download the Mumbai RDS certificate bundle once:

```
curl -o ap-south-1-bundle.pem \
  https://truststore.pki.rds.amazonaws.com/ap-south-1/ap-south-1-bundle.pem

psql "host=essa-prod-db.abcd1234efgh.ap-south-1.rds.amazonaws.com port=5432 \
  dbname=essa user=essa_admin sslmode=verify-full \
  sslrootcert=/path/to/ap-south-1-bundle.pem"
```

You should land at an `essa=>` prompt. If you do not, jump to the troubleshooting table in Section 10 — the failure message tells you exactly which of the four usual causes it is.

4 Create the application's own database role

The master user is for administration. The application gets its own role, so that a leaked application password cannot drop the instance. Still connected as `essa_admin`:

```
-- 1. the role the app logs in as
CREATE ROLE essa_app LOGIN PASSWORD 'a-long-password-with-no-url-special-characters';

-- 2. the app runs create_all() at boot, and creates the "shop" schema itself,
--    so it needs to own the database rather than merely read it
ALTER DATABASE essa OWNER TO essa_app;
GRANT ALL PRIVILEGES ON DATABASE essa TO essa_app;

-- 3. PostgreSQL 15 and later revoke CREATE on the public schema by default
\c essa
GRANT ALL ON SCHEMA public TO essa_app;
ALTER SCHEMA public OWNER TO essa_app;
```

WHY THE APP NEEDS CREATE ON THE DATABASE

On start-up `backend/app/pos_mount.py` runs `CREATE SCHEMA IF NOT EXISTS "shop"`. The warehouse and the retail shop were written as separate SQLite files and four of their table names collide — `categories`, `products`, `stock_movements` and `users`. On one Postgres database, whichever app calls `create_all()` first wins the name and the other then reads a table with its own name and the wrong columns. `users` is the dangerous one: shop logins and warehouse accounts merging is a security problem, not an error message. If the role cannot create that schema, the isolation silently does not happen.

5 Point the application at the database

The app reads real environment variables only — there is no `.env` loading in this backend, so a file dropped beside the code is read by nobody. Set the variable in the host's configuration (EC2 user data, the systemd unit, the ECS task definition, or Vercel's project settings, depending on where the server runs).

```
ESSA_DATABASE_URL=postgresql://essa_app:PASSWORD@essa-prod-db.abcd1234efgh.ap-south-1.rds.amazonaws.com:5432/essa?sslmode=require
```

USE POSTGRESURL:// EXACTLY — NO DRIVER SUFFIX

`postgresql+psycopg2://` and `postgresql+psycopg://` are both valid SQLAlchemy URLs and both will connect. But `_isolate_shop_schema()` in `pos_mount.py` tests `url.startswith(("postgres://", "postgresql://"))` and returns silently when the test fails. A driver suffix therefore means the `shop` schema is never created, the POS tables land in `public` alongside the warehouse's, and the first symptom is a column that "does not exist" on a table that plainly does. The same check governs `/api/status`, which will report the URL as *IGNORED* — not a *postgres URL*. Plain `postgresql://` is correct; `psycopg2` is SQLAlchemy's default driver for it, and it is the one pinned in `requirements.txt`.

Two more things the code does that are worth knowing:

- `ESSA_DATABASE_URL` is the first of five candidate variable names the app checks (`POSTGRES_URL` , `POSTGRES_PRISMA_URL` , `DATABASE_URL` and `POSTGRES_URL_NON_POOLING` follow). Setting the first one means nothing a host injects can quietly outrank it.
- You do **not** set `DATABASE_URL` yourself. The shop reads it, but `_isolate_shop_schema()` overwrites it at boot with the URL the warehouse actually chose, plus `SHOP_DB_SCHEMA=shop` — so both apps are provably on the same database. The `DATABASE_URL=sqlite:///data/textile_shop.db` line in the Dockerfile is a local default and is superseded.

PASSWORD CHARACTERS

If the password contains a URL-special character it must be percent-encoded in the connection string: `@` → `%40` , `:` → `%3A` , `/` → `%2F` , `#` → `%23` , `%` → `%25` . The failure when you forget is *password authentication failed*, which reads like a wrong password. Generating a password from letters and digits only avoids the whole class of problem.

THE OTHER ENVIRONMENT VARIABLES

Set these at the same time — three of them have defaults that are published in the repository:

Variable	Set it to
<code>ESSA_AUTH_SECRET</code>	A long random string. It signs every login token; the repo default is a placeholder, and anyone who has read the source could mint a super-admin token. Generate with <code>python -c "import secrets;print(secrets.token_hex(32))"</code> .
<code>ESSA_SUPERADMIN_PASSWORD</code>	Your own (default <code>super@123</code> is in the repo).
<code>ESSA_ADMIN_PASSWORD</code>	Your own (default <code>essa@123</code>).
<code>ESSA_USER_PASSWORD</code>	Your own (default <code>user@123</code>).
<code>ESSA_STATE_DIR</code>	A writable, persistent path — <code>/data</code> in the container. Invoice images and the vision key live here.
<code>ANTHROPIC_API_KEY</code>	Optional; can also be entered on the settings screen.
<code>ESSA_COMPANY_NAME</code> / <code>ESSA_COMPANY_GSTIN</code>	Essa's details, as they should appear on invoices.

SET THE THREE PASSWORDS BEFORE THE FIRST BOOT

The accounts are seeded on first start and *only if missing* — which is what stops a redeploy reverting a password someone changed in the app. It cuts both ways: boot once with the defaults and setting the variables afterwards changes nothing, because the accounts now exist.

6 Move the existing data off Supabase

Do this with the application stopped, so nothing is written to the old database mid-dump.

```
# 1. Dump from Supabase. Use the DIRECT connection on port 5432 –
# not the transaction pooler on 6543, which pg_dump cannot use.
pg_dump --no-owner --no-privileges --format=custom \
  "postgresql://postgres:PASS@db.PROJECTREF.supabase.co:5432/postgres" \
  -f essa-supabase.dump

# 2. Restore into RDS
pg_restore --no-owner --no-privileges --schema=public --schema=shop \
  -d "postgresql://essa_app:PASS@essa-prod-db.abcd1234efgh.ap-south-1.rds.amazonaws.com:5432/essa?
  sslmode=require" \
  essa-supabase.dump

# 3. Sanity check
psql "$ESSA_DATABASE_URL" -c "\dn" -- expect public + shop
psql "$ESSA_DATABASE_URL" -c "select count(*) from users;" -- warehouse accounts
psql "$ESSA_DATABASE_URL" -c "select count(*) from shop.users;" -- shop logins
```

- Your `pg_dump` must be at least as new as the source server, or it refuses to run.
- Supabase-specific schemas (`auth`, `storage`, `extensions`) are not used by this app and should not be restored — hence the explicit `--schema` flags.
- Uploaded invoice images are files, not rows. They live under `ESSA_STATE_DIR/uploads` and have to be copied across separately (to the EC2 volume, or to S3 if you move to object storage).

7 Verify the whole path

1. **GET `/api/status`** — this endpoint reports which of the five candidate variables it found and the verdict on each (*usable*, *not set*, *IGNORED — still a template*, *IGNORED — not a postgres URL*). Names and verdicts only; the values are passwords. If `ESSA_DATABASE_URL` is not *usable*, stop here.
2. **Log in to the warehouse UI** with the admin account and open Inventory.
3. **Open the retail shop at `/pos`** and log in. Different table, different credentials — this is the check that the `shop` schema separation worked.
4. **Upload one invoice** and confirm the image comes back on the review screen; that exercises `ESSA_STATE_DIR` as well as the database.
5. **Take a manual snapshot** in the RDS console and label it `essa-pre-golive`.

8 Connection pooling and limits

`backend/app/database.py` deliberately uses `NullPool` — one connection per request, handed back at the end — because a serverless deployment runs many short-lived instances and pooled connections would be abandoned rather than reused. On a long-running EC2 or container server that reasoning no longer applies, and a real pool is both faster and gentler on the database:

```
engine = create_engine(
    DB_URL, pool_size=5, max_overflow=10, pool_recycle=1800,
    pool_pre_ping=True, connect_args={"connect_timeout": 10},
)
```

RDS sets `max_connections` from instance memory (`LEAST({DBInstanceClassMemory/9531392}, 5000)`), which works out at roughly:

Instance class	Memory	Approx. max connections
<code>db.t4g.micro</code>	1 GiB	~112
<code>db.t4g.small</code>	2 GiB	~225

Instance class	Memory	Approx. max connections
db.t4g.medium	4 GiB	~450
db.t4g.large	8 GiB	~900

Some of those are reserved for RDS itself. If you ever see *FATAL: too many connections*, the fix is a pool, not a bigger instance.

9 Lock it down before go-live

- **Public access off**, inbound 5432 only from the app's security group.
- **Force SSL**. Create a custom DB parameter group with `rds.force_ssl = 1` and attach it, so an unencrypted client is refused rather than trusted. Reboot required.
- **No passwords in git**. Environment variables on the host, or AWS Secrets Manager. Rotate the application password on a schedule.
- **Deletion protection on**, backup retention at 30 days for production, and one manual snapshot before every deployment.
- **Alarms**. CloudWatch alarms on `FreeStorageSpace`, `CPUUtilization`, `DatabaseConnections` and — for a T-class instance — `CPUCreditBalance`, which is what turns a slow month-end into a surprise on the bill.
- **Restore drill**. Restore a snapshot into a throwaway instance once, before you need it. A backup you have never restored is a hypothesis.

10 Troubleshooting

What you see	What it actually is
connection timed out / hangs	Security group has no inbound rule for your source, or the instance is not publicly accessible and you are outside the VPC. The client is not getting a refusal, which is the signature of a firewall rather than a wrong password.
no pg_hba.conf entry ... no encryption	<code>rds.force_ssl</code> is on and the client did not ask for TLS. Add <code>?sslmode=require</code> .
database "essa" does not exist	"Initial database name" was left blank at creation. Connect to <code>postgres</code> and run <code>CREATE DATABASE essa OWNER essa_app;</code> .
permission denied for schema public	PostgreSQL 15+ default. Run the grants in Step 4.
password authentication failed with a password you know is right	A URL-special character in the password that was not percent-encoded.
POS reports a column that "does not exist" on a table that does	The <code>shop</code> schema was never created — almost always a driver suffix in <code>ESSA_DATABASE_URL</code> (Step 5), sometimes a role without CREATE rights (Step 4).
FATAL: too many connections	<code>NullPool</code> under burst load. Switch to a pooled engine (Section 8).
App crashes at start-up writing files	Not the database. <code>main.py</code> runs <code>create_all()</code> , <code>_migrate()</code> and <code>use rs.seed()</code> at import, and <code>ESSA_STATE_DIR</code> must be writable and persistent.
/api/status says IGNORED — still a template	The value still contains <code><</code> or <code>></code> from a copied example.

11 Appendix — the same thing from the CLI

```
aws rds create-db-instance \  
  --region ap-south-1 \  
  --db-instance-identifier essa-prod-db \  
  --db-name essa \  
  --engine postgres \  
  --engine-version 16.10 \  
  --db-instance-class db.t4g.small \  
  --allocated-storage 100 \  
  --max-allocated-storage 500 \  
  --storage-type gp3 \  
  --storage-encrypted \  
  --master-username essa_admin \  
  --manage-master-user-password \  
  --vpc-security-group-ids sg-0yourdbsecuritygroup \  
  --db-subnet-group-name essa-db-subnets \  
  --backup-retention-period 7 \  
  --no-publicly-accessible \  
  --deletion-protection \  
  --auto-minor-version-upgrade \  
  --enable-performance-insights  
  
# watch until it reports "available"  
aws rds describe-db-instances --region ap-south-1 \  
  --db-instance-identifier essa-prod-db \  
  --query 'DBInstances[0].[DBInstanceStatus,Endpoint.Address]' --output text
```

`--manage-master-user-password` hands the master password to AWS Secrets Manager, so it never appears in your shell history or a script.

Prepared by Phoenixx IT for Essa Garments Private Limited. Console screens and defaults reflect AWS as of 4 September 2026. Commands are given for the Mumbai (ap-south-1) region throughout. Companion document: *Essa on AWS — Plans & Pricing*.